پروژه پایانی دوره: از ایده تا ساخت نرم‌افزار با پایتون و هوش مصنوعی

۱. هدف پروژه پایانی

در این دوره با مفاهیم اصلی پایتون، از متغیرها و انواع داده تا شرط‌ها، حلقه‌ها، توابع، ساختارهای داده، ماژول‌ها، مدیریت فایل‌ها و برنامه‌نویسی شیءگرا آشنا شدیم. همچنین یاد گرفتیم چگونه از هوش مصنوعی برای درک مسئله، برنامه‌ریزی، تولید و بازبینی کد و رفع خطا کمک بگیریم.

اکنون زمان آن رسیده است که این آموخته‌ها را در قالب یک پروژه واقعی‌تر کنار هم قرار دهیم.

هدف پروژه پایانی صرفاً نوشتن چند خط کد نیست. باید بتوانید یک مسئله مشخص را شناسایی کنید، راه‌حلی برای آن طراحی کنید، راه‌حل را به بخش‌های کوچک‌تر تقسیم کنید و در نهایت برنامه‌ای بسازید که بتواند وظیفه موردنظر را انجام دهد.

در این مسیر، هوش مصنوعی می‌تواند نقش یک دستیار را داشته باشد؛ اما تصمیم‌گیری درباره نیازمندی‌ها، انتخاب راه‌حل، بررسی رفتار برنامه و اطمینان از درستی آن همچنان بر عهده شماست.

مهم‌ترین دستاورد این پروژه آن است که بتوانید فرایند تبدیل یک ایده به یک نرم‌افزار قابل استفاده را تجربه کنید.

۲. انتخاب پروژه

در ادامه هفت پروژه پیشنهادی معرفی شده‌اند. هر پروژه نقطه شروع مشخصی دارد و می‌توان آن را در چند مرحله توسعه داد.

لازم نیست تمام قابلیت‌های پیشرفته یک پروژه را پیاده‌سازی کنید. ابتدا باید نسخه‌ای کوچک اما قابل استفاده بسازید و سپس، در صورت داشتن زمان و توانایی کافی، امکانات بیشتری به آن اضافه کنید.

پروژه‌ها از نظر میزان پیچیدگی متفاوت‌اند. بهتر است پیش از انتخاب، به سه سؤال پاسخ دهید:

- آیا ایده پروژه برای من جذاب است؟
- آیا می‌توانم نسخه اولیه آن را با مفاهیمی که یاد گرفته‌ام بسازم؟
- آیا برای توسعه بیشتر آن ایده‌هایی دارم؟

انتخاب پروژه‌ای که به آن علاقه دارید، معمولاً انگیزه شما را برای آزمایش، رفع خطا و یادگیری بیشتر افزایش می‌دهد.

پروژه‌های پیشنهادی

شماره	پروژه عنوان	تقریبی سختی	اصلی ابزار
۱	عبور رمز قدرت آزمون	آسان	Tkinter و Python
۲	آموزشی کارت‌فلش	آسان	Tkinter و Python
۳	مطالعه و تمرکز تایمر	متوسط تا آسان	Tkinter و Python
۴	تایپ سرعت آزمون	متوسط تا آسان	Tkinter و Python
۵	قیچی کاغذ، سنگ، بازی	آسان بسیار	Tkinter و Python
۶	عمومی اطلاعات مسابقه بازی	آسان	Tkinter و Python
۷	عدد حدس بازی	آسان بسیار	Tkinter و Python

در این پروژه‌ها از Tkinter برای ساخت رابط گرافیکی استفاده می‌کنیم. این کتابخانه امکان ساخت پنجره، دکمه، کادر ورود اطلاعات، برچسب متنی و سایر اجزای ساده رابط کاربری را فراهم می‌کند.

ممکن است برای بعضی قابلیت‌ها به ماژول‌های دیگری از پایتون نیز نیاز داشته باشید؛ برای مثال، ماژول `random` برای تولید مقادیر تصادفی یا ماژول‌های مرتبط با زمان برای اندازه‌گیری مدت یک فعالیت.

لازم نیست پیش از شروع، تمام جزئیات ابزارهای موردنیاز را بدانید. بخشی از فرایند توسعه نرم‌افزار، شناسایی ابزار مناسب در زمان نیاز است.

---

۳. پیش از کدنویسی چه کارهایی انجام دهیم؟

یکی از اشتباه‌های رایج در شروع پروژه این است که بدون مشخص کردن هدف و قابلیت‌های برنامه، مستقیماً نوشتن کد را آغاز کنیم.

این روش ممکن است برای یک تمرین بسیار کوچک جواب بدهد؛ اما هرچه پروژه بزرگ‌تر شود، احتمال سردرگمی، دوباره‌کاری و ایجاد خطا افزایش می‌یابد.

پیش از شروع کدنویسی، بهتر است مسئله را بررسی و مسیر کلی حل آن را مشخص کنید.

این مرحله را می‌توان پیش‌تولید یا طراحی اولیه پروژه نامید.

مرحله اول: مسئله را تعریف کنید

ابتدا به این سؤال پاسخ دهید:

این برنامه قرار است چه مشکلی را حل کند یا چه کاری را برای کاربر انجام دهد؟

برای مثال، در پروژه فلش‌کارت آموزشی، مسئله این است که کاربر بتواند مجموعه‌ای از سؤال‌ها و جواب‌ها را مرور کند و متوجه شود چه تعداد از کارت‌ها را پشت سر گذاشته است.

تعریف مسئله باید روشن و محدود باشد. جمله‌ای مانند «می‌خواهم یک برنامه آموزشی بسازم» به اندازه کافی دقیق نیست؛ اما «می‌خواهم برنامه‌ای بسازم که سؤال‌های آموزشی را یکی‌یکی نمایش دهد، امکان دیدن پاسخ را فراهم کند و میزان پیشرفت را نشان دهد» نقطه شروع بهتری است.

مرحله دوم: کاربر و نحوه استفاده از برنامه را مشخص کنید

مشخص کنید چه کسی قرار است از برنامه استفاده کند و برای انجام کار خود چه مراحلی را طی خواهد کرد.

برای مثال، در بازی حدس عدد، کاربر برنامه را باز می‌کند، عددی وارد می‌کند، راهنمایی می‌گیرد و تا رسیدن به پاسخ یا پایان فرصت بازی به تلاش خود ادامه می‌دهد.

توصیف این مسیر به شما کمک می‌کند بفهمید برنامه به چه ورودی‌ها، پردازش‌ها و خروجی‌هایی نیاز دارد.

مرحله سوم: ورودی، پردازش و خروجی را مشخص کنید

برای هر پروژه سه بخش را در نظر بگیرید:

ورودی: اطلاعاتی که کاربر وارد می‌کند یا برنامه دریافت می‌کند.

پردازش: تصمیم‌گیری‌ها و محاسباتی که برنامه انجام می‌دهد.

خروجی: نتیجه‌ای که برنامه به کاربر نشان می‌دهد.

برای نمونه، در آزمون سرعت تایپ:

- ورودی: متنی که کاربر تایپ می‌کند.
- پردازش: مقایسه متن واردشده با متن هدف، محاسبه مدت زمان و ارزیابی عملکرد.
- خروجی: سرعت تایپ، میزان دقت و نتیجه آزمون.

این روش کمک می‌کند مسئله‌ای که در ابتدا کلی به نظر می‌رسد، به اجزای مشخص‌تری تقسیم شود.

مرحله چهارم: نسخه اولیه را تعریف کنید

پیش از طراحی قابلیت‌های پیشرفته، مشخص کنید ساده‌ترین نسخه قابل استفاده برنامه چه ویژگی‌هایی دارد.

به این نسخه اولیه می‌توان MVP یا حداقل محصول پذیرفتنی گفت.

برای مثال، نسخه اولیه بازی سنگ، کاغذ، قیچی به انتخاب کاربر، انتخاب تصادفی کامپیوتر، تعیین برنده و نمایش نتیجه نیاز دارد. ثبت تاریخچه بازی‌ها یا افزودن انیمیشن را می‌توان به مراحل بعد موکول کرد.

هدف این است که ابتدا یک برنامه کوچک اما کامل بسازید، نه یک برنامه بزرگ با قابلیت‌های متعدد که هیچ‌کدام به‌درستی کار نمی‌کنند.

مرحله پنجم: مسیر کاربر را طراحی کنید

مشخص کنید کاربر از لحظه باز کردن برنامه تا پایان کار چه مراحلی را طی خواهد کرد.

می‌توانید مسیر را با یک فهرست ساده، نمودار جریان یا شبه‌کد توضیح دهید.

برای مثال، مسیر کلی یک بازی می‌تواند چنین باشد:

۱. شروع بازی؛
۲. دریافت انتخاب کاربر؛
۳. بررسی انتخاب؛
۴. تعیین نتیجه؛
۵. به‌روزرسانی امتیاز؛
۶. نمایش نتیجه؛
۷. ادامه بازی یا پایان آن.

این توضیح هنوز کد برنامه نیست. هدف آن مشخص کردن منطق و ترتیب عملیات است.

مرحله ششم: ساختار داده‌ها را انتخاب کنید

اطلاعات موردنیاز برنامه را بررسی کنید و مشخص کنید هر نوع داده چگونه باید نگهداری شود.

بسته به پروژه، ممکن است به این موارد نیاز داشته باشید:

- متغیرها برای نگهداری مقدارهایی مانند امتیاز و زمان باقی‌مانده؛
- رشته‌ها برای نگهداری متن سؤال‌ها یا پاسخ‌ها؛
- لیست‌ها برای نگهداری مجموعه‌ای از کارت‌ها یا سؤال‌ها؛
- دیکشنری‌ها برای نگهداری اطلاعات مرتبط با هر سؤال یا کارت؛
- توابع برای جدا کردن عملیات مختلف؛
- کلاس‌ها در صورتی که سازمان‌دهی داده‌ها و رفتارهای مرتبط با یکدیگر به این روش کمک کند.

لازم نیست برای هر پروژه از تمام این ابزارها استفاده کنید. انتخاب ساختار مناسب به مسئله بستگی دارد.

مرحله هفتم: رابط کاربری را روی کاغذ طراحی کنید

پیش از ساخت پنجره گرافیکی، یک طرح ساده از ظاهر برنامه بکشید یا با استفاده از ابزارهای طراحی، چیدمان آن را مشخص کنید.

مشخص کنید چه بخش‌هایی در پنجره وجود خواهند داشت، کاربر از کجا اطلاعات وارد می‌کند و نتیجه کجا نمایش داده می‌شود.

این طرح می‌تواند بسیار ساده باشد. هدف آن نیست که از ابتدا یک رابط کاربری زیبا و حرفه‌ای داشته باشید؛ هدف این است که پیش از کدنویسی، درباره نحوه تعامل کاربر با برنامه تصمیم بگیرید.

---

۴. چگونه پروژه را به بخش‌های کوچک‌تر تقسیم کنیم؟

پس از طراحی اولیه، باید کار را به وظایفی تقسیم کنید که بتوانید هرکدام را جداگانه پیاده‌سازی و آزمایش کنید.

برای مثال، یک برنامه آموزشی ممکن است به این وظایف تقسیم شود:

- آماده‌سازی اطلاعات آموزشی؛
- نمایش سؤال؛
- نمایش پاسخ؛
- حرکت به کارت بعدی؛
- محاسبه میزان پیشرفت؛
- ساخت رابط کاربری؛
- مدیریت وضعیت پایان فعالیت.

این تقسیم‌بندی کمک می‌کند بدانید در هر مرحله دقیقاً چه چیزی را می‌سازید.

بهتر است ابتدا بخش اصلی منطق برنامه را مشخص کنید و سپس آن را به رابط گرافیکی متصل کنید. این کار همیشه یک قانون اجباری نیست، اما برای پروژه‌های مقدماتی معمولاً روش مناسبی است.

در طول توسعه، هر بخش را به‌صورت جداگانه بررسی کنید. اگر پس از اضافه کردن یک قابلیت جدید برنامه دچار مشکل شد، این کار پیدا کردن منشأ خطا را آسان‌تر می‌کند.

از توابع برای سازمان‌دهی برنامه استفاده کنید

اگر چند عملیات مستقل در برنامه وجود دارد، بررسی کنید آیا می‌توان آن‌ها را در قالب توابع جداگانه سازمان‌دهی کرد.

برای مثال، عملیات آماده‌سازی یک دور بازی، بررسی انتخاب کاربر، محاسبه امتیاز و به‌روزرسانی نمایشگر می‌توانند مسئولیت‌های جداگانه‌ای داشته باشند.

هدف استفاده از تابع به‌خاطر خود تابع نیست. تابع زمانی مفید است که مسئولیت مشخصی داشته باشد، استفاده مجدد از یک عملیات را آسان کند یا خواندن و آزمایش برنامه را ساده‌تر سازد.

اگر منطق برنامه واقعاً به کلاس‌ها و اشیا نیاز پیدا کند، می‌توانید از برنامه‌نویسی شیءگرا استفاده کنید؛ اما برای کوچک نگه داشتن پروژه، خودتان را مجبور به استفاده از کلاس‌ها نکنید.

---

۵. چگونه از هوش مصنوعی در فرایند توسعه کمک بگیریم؟

در این پروژه قرار نیست هوش مصنوعی فقط ابزاری برای تولید سریع کد باشد. می‌توانید از آن در مراحل مختلف توسعه نرم‌افزار استفاده کنید.

نکته مهم این است که پاسخ هوش مصنوعی یک پیشنهاد برای بررسی است، نه تضمینی برای درستی برنامه.

کاربرد اول: بررسی و دقیق‌تر کردن ایده

اگر ایده پروژه هنوز برایتان مبهم است، از هوش مصنوعی بخواهید به شما در مشخص کردن کاربران، مسئله، ورودی‌ها، پردازش‌ها و خروجی‌ها کمک کند.

برای مثال، از آن بخواهید نیازمندی‌های یک برنامه فلش‌کارت را به قابلیت‌های ضروری و قابلیت‌های توسعه‌ای تقسیم کند.

خروجی این مرحله باید به شما در تصمیم‌گیری کمک کند، نه اینکه بدون بررسی به فهرست پیشنهادی وابسته شوید.

کاربرد دوم: طراحی مراحل توسعه

پس از تعریف مسئله، از هوش مصنوعی بخواهید مسیر ساخت برنامه را به وظایف کوچک و قابل آزمایش تقسیم کند.

در درخواست خود توضیح دهید که در حال یادگیری پایتون هستید و می‌خواهید راهنمایی مرحله‌به‌مرحله دریافت کنید، نه یک پروژه کامل و آماده.

بهتر است برای هر وظیفه، ورودی، خروجی موردانتظار و راه آزمایش آن را مشخص کنید.

کاربرد سوم: توضیح مفاهیم و انتخاب ابزار

گاهی هنگام توسعه متوجه می‌شوید که برای یک قابلیت به مفهومی نیاز دارید که هنوز به‌خوبی آن را یاد نگرفته‌اید.

در این شرایط، می‌توانید از هوش مصنوعی بخواهید مفهوم موردنظر را با مثال ساده توضیح دهد، گزینه‌های مختلف را مقایسه کند یا دلیل مناسب بودن یک ابزار را شرح دهد.

در صورت امکان، ابتدا توضیح را بفهمید و سپس آن را در پروژه به کار ببرید. استفاده از ابزاری که دلیل انتخاب و شیوه کار آن را نمی‌دانید، می‌تواند در مراحل بعدی مشکلات بیشتری ایجاد کند.

کاربرد چهارم: توسعه تدریجی کد

پس از تقسیم پروژه به وظایف کوچک، هر بار روی یک بخش تمرکز کنید.

می‌توانید از هوش مصنوعی بخواهید درباره ساختار یک تابع، نحوه تعامل اجزای رابط کاربری یا منطق یک عملیات پیشنهاد بدهد.

هنگام درخواست کمک، مشخص کنید:

- در حال ساخت کدام بخش هستید؛
- این بخش چه وظیفه‌ای دارد؛
- چه ورودی و خروجی‌ای انتظار دارید؛
- چه چیزهایی را قبلاً پیاده‌سازی کرده‌اید؛
- چه محدودیت‌هایی دارید.

درخواست روشن معمولاً به پاسخی مرتبط‌تر منجر می‌شود.

از تولید یک‌باره تمام برنامه، به‌ویژه زمانی که هنوز منطق آن را درک نکرده‌اید، پرهیز کنید. بررسی و اصلاح تدریجی باعث می‌شود کنترل بیشتری بر روند توسعه داشته باشید.

کاربرد پنجم: رفع خطا و بازبینی برنامه

اگر برنامه درست کار نمی‌کند، ابتدا پیام خطا و رفتار مشاهده‌شده را بررسی کنید.

سپس از هوش مصنوعی کمک بگیرید تا علت‌های احتمالی را شناسایی کنید و روش آزمایش هرکدام را توضیح دهد.

بهتر است فقط کپی کردن یک خطای طولانی و پذیرفتن اولین پاسخ را به روش همیشگی خود تبدیل نکنید. شرح دهید که انتظار چه رفتاری داشتید، چه اتفاقی افتاده است و آخرین تغییری که اعمال کرده‌اید چه بوده است.

پس از اصلاح پیشنهادی، برنامه را دوباره اجرا کنید و بررسی کنید که آیا مشکل واقعاً برطرف شده است یا خیر.

کاربرد ششم: طراحی آزمون

از هوش مصنوعی بخواهید حالت‌های معمول، حالت‌های مرزی و ورودی‌های نامعتبر را برای بخش موردنظر شما پیشنهاد کند.

برای مثال، در برنامه‌ای که از کاربر عدد دریافت می‌کند، باید مشخص کنید رفتار برنامه در برابر ورودی خالی، عدد خارج از محدوده یا متن غیرعددی چگونه خواهد بود.

همه پیشنهادها را کورکورانه اجرا نکنید. موارد مرتبط را انتخاب کنید و خودتان نتیجه آن‌ها را بررسی کنید.

چند اصل مهم در استفاده از AI

۱. ابتدا مسئله را بفهمید. حتی اگر برای حل آن از هوش مصنوعی کمک می‌گیرید، باید بتوانید وظیفه‌ای را که قرار است انجام شود توضیح دهید.

۲. پاسخ‌ها را آزمایش کنید. برنامه‌ای که تولید شده اما اجرا نشده است، هنوز یک راه‌حل تأییدشده نیست.

۳. درباره کد سؤال بپرسید. اگر بخشی از کد را نمی‌فهمید، از AI بخواهید نقش هر قسمت را توضیح دهد.

۴. تغییرها را تدریجی انجام دهید. اضافه کردن چند قابلیت به‌صورت هم‌زمان، تشخیص منشأ خطا را دشوارتر می‌کند.

۵. با هر خطا دوباره طراحی را بررسی کنید. گاهی مشکل از یک خط کد نیست؛ ممکن است تقسیم وظایف یا طراحی اولیه مناسب نباشد.

۶. محدودیت‌های AI را در نظر بگیرید. ممکن است کد ناسازگار با محیط اجرا، پیچیده‌تر از نیاز پروژه یا دارای خطاهای منطقی تولید کند.

۷. مسئولیت نتیجه نهایی را بپذیرید. شما باید بتوانید نحوه کار برنامه خود را توضیح دهید و نشان دهید که قابلیت‌های اصلی آن را آزمایش کرده‌اید.

---

۶. معرفی پروژه‌ها و راهنمای توسعه هرکدام

پروژه اول: آزمون قدرت رمز عبور